



Instituto Tecnológico de Costa Rica

Proyecto #3

Estudiante:

Quiriat Mata Araya – 2023379891

Elder Pérez León - 2023166120

Profesor:

Allan Rodríguez Dávila

Curso:

Compiladores e Interpretes

G60

Semestre I 2026

Manual de usuario	1
Instrucciones s de compilación, ejecución y uso.....	1
Requisitos para Ejecución y traducción.....	2
Compilación desde terminal visual estudio	2
Flujo general de ejecución del proyecto	2
Requisitos para Ejecución en QTspin	4
Ejecución	4
Pruebas de funcionalidad	5
Prueba 1	5
Prueba 2	5
Prueba 3	5
4. Descripción del Problema	5
Alcance de la Fase	5
5. Librerías Usadas.....	5
java.io - Entrada/Salida de Archivos	5
java.util - Estructuras de Datos	5
java.util.regex - Análisis de Patrones	5
StringBuilder Construcción Eficiente del Código	5
Resumen de Dependencias	5
6. Análisis de resultados	5
7. Detalle de los Algoritmos más Importantes del Proyecto	5
Algoritmo Principal de Traducción - método traducir()	5
Algoritmo de Traducción de Expresiones	5
Algoritmo de Gestión del Frame de Activación	5
Inicio de función	5
Cuerpo (variables locales y temporales)	5
Retorno	5
Algoritmo de Llamada a Función — Escribir_llamada()	5
Algoritmo de Acceso a Arreglos Bidimensionales	5
Algoritmo de Comparación Flotante	5
Subrutinas de Potenciación	5
potencia_int (base entera, exponente entero)	5
potencia_float (base flotante, exponente flotante)	5

Manual de usuario

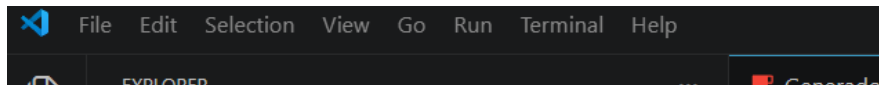
Instrucciones de compilación, ejecución y uso.

Requisitos para Ejecución y traducción

1. Para compilar y ejecutar el proyecto se requiere:
2. Tener instalado **Java JDK**, preferiblemente Java
3. Visual Studio Code.
4. Contar con una terminal o consola para ejecutar comandos.
5. Tener acceso a la carpeta del proyecto
 - 5.1. Abrir **Visual Studio Code**.
 - 5.2. Seleccionar la opción **File > Open Folder**.
 - 5.3. Buscar y abrir la carpeta con el proyecto.

Compilación desde terminal visual estudio

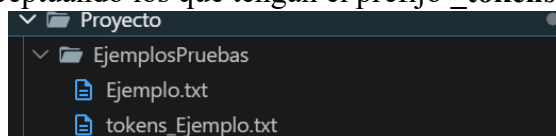
1. En la pestaña superior de visual estudio ubique el botón de terminal y abra la.



2. Ubicarse en la carpeta principal del proyecto

```
cd Proyecto
```

3. Dentro del proyecto ubique la carpeta Proyecto -> ejemplosPruebas, dentro de esta carpeta puede crear un nuevo txt con un código fuente o pegar alguno que desee.
Nota: el proyecto hace la ejecución de todos los archivos que se encuentren dentro de dicha carpeta exceptuando los que tengan el prefijo **_tokens**.



4. Luego compilar los archivos Java

```
javac -cp ".;lib/jflex.jar;lib/java-cup-runtime.jar" src/app/Main.java
```

5. Ejecutar el proyecto

```
java -cp ".;lib/jflex.jar;lib/java-cup-runtime.jar;src/app" Main
```

Flujo general de ejecución del proyecto

Al terminar de correr el proyecto en la parte de traducción vera lo siguiente en la terminal.

```

Archivo: Ejemplo.txt

[ LÉXICO ]
Sin errores léxicos.
[ TABLA DE SÍMBOLOS ]
[ TABLA DE SÍMBOLOS ]
Scope: [ global ]
TOKEN      TIPO      TIPO_DATO  FILA      COLUMNA
(sin identificadores)
Scope: [ __main__ ]
TOKEN      TIPO      TIPO_DATO  FILA      COLUMNA
base        VARIABLE   int        2         11
exp         VARIABLE   int        3         11
resultado   VARIABLE   int        4         11
[ PARSER ]
Sin errores sintácticos.
[ ERRORES SEMÁNTICOS ]
Sin errores semánticos.
[ CÓDIGO INTERMEDIO ]

Código guardado en: generated/codigo30_Ejemplo.txt
Código MIPS generado en: generated/Ejemplo.asm
El archivo cumple con la gramática y puede ser procesado.

```

La terminal si su código fuente está libre de errores, se le indicara que su archivo es válido y libre de errores ahora bien a continuación se mostrara lo que se espera en caso de errores.

```

Archivo: Ejemplo.txt

[ LÉXICO ]
Sin errores léxicos.
Error sintáctico en línea 2, columna 22: token inesperado '!'
Error sintáctico en línea 6, columna 5: token inesperado 'resultado'
[ TABLA DE SÍMBOLOS ]
[ TABLA DE SÍMBOLOS ]
Scope: [ global ]
TOKEN      TIPO      TIPO_DATO  FILA      COLUMNA
(sin identificadores)
Scope: [ __main__ ]
TOKEN      TIPO      TIPO_DATO  FILA      COLUMNA
exp         VARIABLE   int        3         11
[ PARSER ]
Errores sintácticos encontrados: 4
Error sintáctico en línea 2, columna 22: token inesperado '!'
  + Recuperado: se omitió sentencia inválida hasta '!'
Error sintáctico en línea 6, columna 5: token inesperado 'resultado'
  + Recuperado: se omitió sentencia inválida hasta '!'
[ ERRORES SEMÁNTICOS ]
Error semántico en línea 3, columna 11: no se puede inicializar la variable 'exp' de tipo 'int' con una expresión de tipo 'float'.
Variable no declarada 'resultado' en línea 7, columna 13
[ CÓDIGO INTERMEDIO ]
No se genera código intermedio por presencia de errores.
Errores léxicos: 0
Errores sintácticos: 4
Errores semánticos: 2

```

El propio compilador le indicara sus errores y la fila, columna que debe corregir según la gramática.

Requisitos para Ejecución en QTspin

1. Tener instalado **QtSpin** en la computadora.
2. Contar con el archivo MIPS generado por el proyecto, normalmente con extensión .asm o
3. Verificar que el código fuente analizado no tenga errores léxicos, sintácticos ni semánticos antes de generar el código MIPS.
4. Confirmar que el archivo generado contenga las secciones básicas de MIPS:

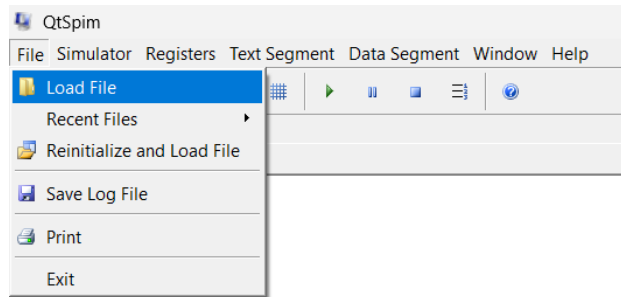
```
.data
    newline:                .asciiz      "\n"

    .text
    .globl main
```

Ejecución

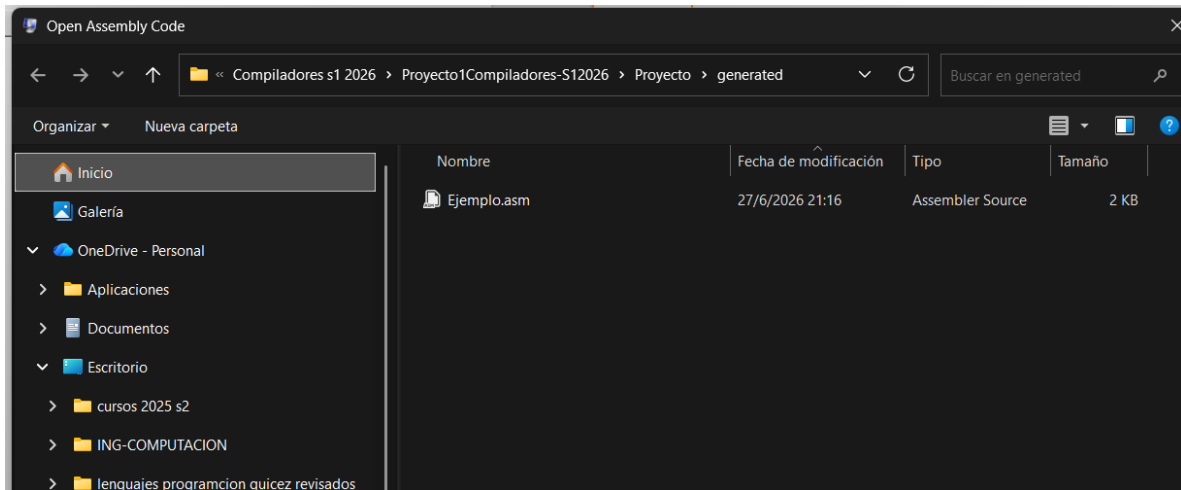
1. Abrir el programa **QtSpin**.
2. Seleccionar la opción:

File > Load File

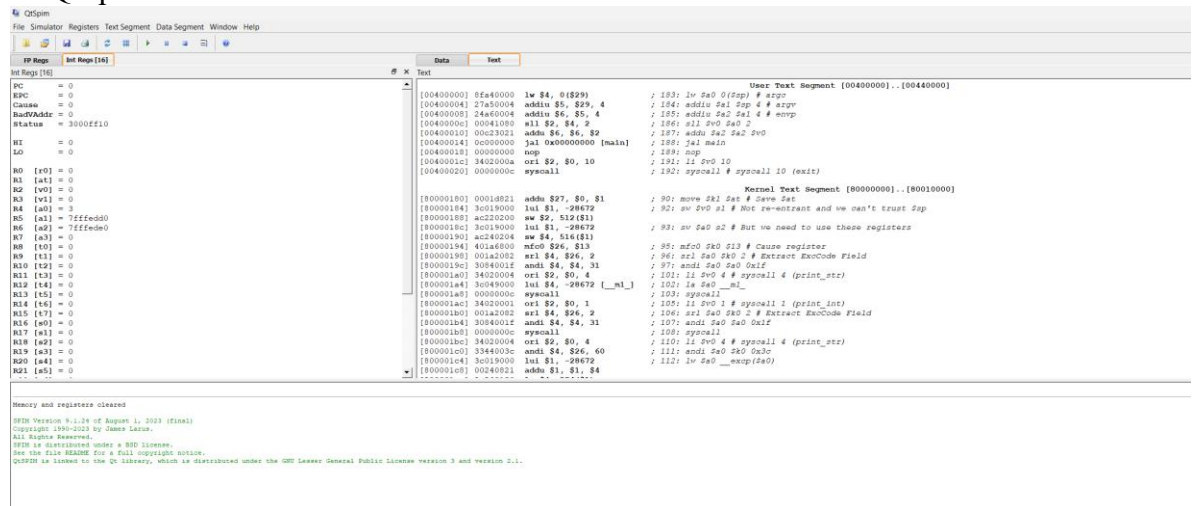


3. Buscar y seleccionar el archivo MIPS generado por el proyecto, por ejemplo:

Ejemplo.asm



- Una vez cargado el archivo, revisar que no aparezcan errores en la consola inferior de QtSpim.



- Para iniciar la ejecución, presionar el botón: Run (triángulo verde)



- Si el programa contiene instrucciones de salida, el resultado se mostrará en la consola de QtSpim.

The image shows a Qt application interface. The top window, titled 'Text', displays MIPS assembly code. The bottom window, titled 'Console', shows the output of the program's execution.

Assembly Code (Text Window):

```

[00400b94] 34080000 ori $8, $0, 0 ; 703: li $t0, 0
[00400b98] 27bdfffc addiu $29, $29, -4 ; 704: addiu $sp, $sp, -4
[00400b9c] afc8fed0 sw $8, -304($30) ; 705: sw $t0, -304($fp)
[00400ba0] 8fc4fed0 lw $4, -304($30) ; 706: lw $a0, -304($fp)
[00400ba4] 0c100076 jal 0x004001d8 [fib] ; 707: jal fib
[00400ba8] 27bdfffc addiu $29, $29, -4 ; 708: addiu $sp, $sp, -4
[00400bac] afc2fecc sw $2, -308($30) ; 709: sw $v0, -308($fp)
[00400bb0] 27bdfffc addiu $29, $29, -4 ; 710: addiu $sp, $sp, -4 # fibCero (int) @ -312($fp)
[00400bb4] 8fc8fecc lw $8, -308($30) ; 711: lw $t0, -308($fp)
[00400bb8] afc8fec8 sw $8, -312($30) ; 712: sw $t0, -312($fp)
[00400bc4] 3c011001 lui $1, 1007 [str_0] ; 713: la $t0, str_0

```

Console Output:

```

[00400b94] factorial de
[00400b98] 6
[00400b9c] 720
[00400ba0] fibonacci de
[00400ba4] 7
[00400ba8] 13
[00400bac] chequeo1 deberia ser true
[00400bb0] 1
[00400bb4] chequeo2 deberia ser false (impar)
[00400bb8] 0
[00400bc4] chequeo3 deberia ser false (fuera de rango)
[00400bc8] 0
[00400bd0] condicionCompuesta deberia ser true
[00400bd4] 0
[00400bd8] condicion mixta con llamadas recursivas en if: true
[00400be0] factorial de 0 deberia ser 1
[00400be4] 1
[00400be8] fib de 0 deberia ser 0
[00400bf0] 0
[00400bf4] 0
[00400bf8] 0

```

Public License version

Pruebas de funcionalidad

Prueba 1

Ejecucion de código con varios eventos salida exitosa en qtSpin y consola.

GeneradorMIPS.java

Main.java 3

Ejemplo.asm M

Ejemplo.txt M X

Proyecto > EjemplosPruebas > Ejemplo.txt

```

43 | int ~ resultadoFactorial <-
44 | cout <| "factorial de" |>|
45 | cout <| x |>|
46 | cout <| resultadoFactorial |>|
47 |
48 | int ~ n <- 7!
49 | int ~ resultadoFib <- fib <| n |>|
50 | cout <| "fibonacci de" |>|
51 | cout <| n |>|
52 | cout <| resultadoFib |>|
53 |
54 | bool ~ chequeo1 <- esParEnRango <| 8, 1, 10 |>|
55 | cout <| "chequeo1 deberia ser true" |>|
56 | cout <| chequeo1 |>|
57 |
58 | bool ~ chequeo2 <- esParEnRango <| 7, 1, 10 |>|
59 | cout <| "chequeo2 deberia ser false (impar)" |>|
60 | cout <| chequeo2 |>|
61 |
62 | bool ~ chequeo3 <- esParEnRango <| 20, 1, 10 |>|
63 | cout <| "chequeo3 deberia ser false (fuera de rango)" |>|
64 | cout <| chequeo3 |>|
65 |
66 | int ~ a <- 5!
67 | int ~ b <- 10!
68 | int ~ c <- 15!
69 |
70 | bool ~ condicionCompuesta <- $<|equal<| a, b |>|> @ greather_t<| c, b |>| # less_t<| a
71 |
72 | cout <| "condicionCompuesta deberia ser true" |>|
73 | cout <| condicionCompuesta |>|
74 |
75 | if <| greather_t<| factorial <| 3 |>, 5 |>| @ less_t<| fib <| 5 |>, 100 |>|
76 | |
77 | | cout <| "condicion mixta con llamadas recursivas en if: true" |>|
78 | |
79 | else
80 | |
81 | | cout <| "condicion mixta con llamadas recursivas en if: false" |>|
82 | |
83 |
84 | int ~ baseRecursionCero <- factorial <| 0 |>|
85 | cout <| "factorial de 0 deberia ser 1" |>|
86 | cout <| baseRecursionCero |>|
87 |
88 | int ~ fibCero <- fib <| 0 |>|
89 | cout <| "fib de 0 deberia ser 0" |>|
90 | cout <| fibCero |>|
91 |

```

f1

VARIABLE

int

25

15

f2

VARIABLE

int

26

15

Scope: [Funcion esParEnRango]

TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
valor	PARAMETRO	int	31	30
minimo	PARAMETRO	int	31	43
maximo	PARAMETRO	int	31	57
dentroDeRango	VARIABLE	bool	33	12
residuo	VARIABLE	int	34	11
esPar	VARIABLE	bool	35	12
resultadoFinal	VARIABLE	bool	36	12

Scope: [__main__]

TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
x	VARIABLE	int	42	11
resultadoFactorial	VARIABLE	int	43	11
n	VARIABLE	int	48	11
resultadoFib	VARIABLE	int	49	11
chequeo1	VARIABLE	bool	54	12
chequeo2	VARIABLE	bool	58	12
chequeo3	VARIABLE	bool	62	12
a	VARIABLE	int	66	11
b	VARIABLE	int	67	11
c	VARIABLE	int	68	11
condicionCompuesta	VARIABLE	bool	70	11
baseRecursionCero	VARIABLE	int	84	11
fibCero	VARIABLE	int	88	11

Scope: [if __main__]

TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
(sin identificadores)				

Scope: [else __main__]

TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
(sin identificadores)				

[PARSER]

Sin errores sintácticos.

[ERRORES SEMÁNTICOS]

Sin errores semánticos.

[CÓDIGO INTERMEDIO]

Código guardado en: generated/Codigo3D_Ejemplo.txt

Console

```

factorial de
6
720
fibonacci de
7
13
chequeo1 deberia ser true
1
chequeo2 deberia ser false (impar)
0
chequeo3 deberia ser false (fuera de rango)
0
condicionCompuesta deberia ser true
0
condicion mixta con llamadas recursivas en if: true
factorial de 0 deberia ser 1
1
fib de 0 deberia ser 0
0

```

Prueba 2

código con errores de ejecución, no es posible correr en Qt spin.

```

43 int ~ resultadoFactorial <- 1
44 cout << "factorial de" >> n >> endl
45 cout << x >> endl
46 cout << resultadoFactorial >> endl
47
48 int ~ n <- 7!
49 int ~ resultadoFib <- fib << n >> endl
50 cout << "fibonacci de" >> n >> endl
51 cout << n >> endl
52 cout << resultadoFib >> endl
53
54 bool ~ chequeo1 <- esParEnRango << 8, 1, 10 >> endl
55 cout << "chequeo1 debería ser true" >> endl
56 cout << chequeo1 >> endl
57
58 bool ~ chequeo2 <- esParEnRango << 1, 1, 10 >> endl
59 cout << "chequeo2 debería ser false (impar)" >> endl
60 cout << chequeo2 >> endl
61
62 bool ~ chequeo3 <- esParEnRango << 20, 1, 10 >> endl
63 cout << "chequeo3 debería ser false (fuera de rango)" >> endl
64 cout << chequeo3 >> endl
65
66 int ~ a <- 5!
67 int ~ b <- 10
68 int ~ c <- 15!
69
70 bool ~ condicionCompuesta <- $(equal << a, b >> |> @ greater_t << c, b >> # less_t << a
71
72 cout << "condicionCompuesta debería ser true" >> endl
73 cout << condicionCompuesta >> endl
74
75 if << greater_t << factorial << 3 >>, 5 >> @ less_t << fib << 5 >>, 100 >> |>
76 |:
77 | cout << "condicion mixta con llamadas recursivas en if: true" >> endl
78 |:
79 else
80 |:
81 | cout << "condicion mixta con llamadas recursivas en if: false" >> endl
82 |:
83
84 int ~ baseRecursionCero <- factorial << 0 >> endl
85 cout << "factorial de 0 debería ser 1" >> endl
86 cout << baseRecursionCero >> endl
87
88 int ~ fibCero <- fib << 0 >> endl
89 cout << "fib de 0 debería ser 0" >> endl
90 cout << fibCero >> endl
91 :|

```

TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
x	VARIABLE	int	42	11
resultadoFactorial	VARIABLE	int	43	11
n	VARIABLE	int	48	11
resultadoFib	VARIABLE	int	49	11
chequeo1	VARIABLE	bool	54	12
chequeo3	VARIABLE	bool	62	12
a	VARIABLE	int	66	11
condicionCompuesta	VARIABLE	bool	70	11
baseRecursionCero	VARIABLE	int	84	11
fibCero	VARIABLE	int	88	11

Scope: [__main__]

TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
(sin identificadores)				

Scope: [else__main__]

TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
(sin identificadores)				

[PARSE]

Errores sintácticos encontrados: 4

- Error sintáctico en línea 59, columna 40: token inesperado ',',
→ Recuperado: se omitió sentencia inválida hasta '!'
- Error sintáctico en línea 68, columna 5: token inesperado 'int'
→ Recuperado: se omitió sentencia inválida hasta '!'

[ERRORES SEMÁNTICOS]

Variable no declarada 'chequeo2' en línea 60, columna 13

Variable no declarada 'b' en línea 70, columna 47

[CÓDIGO INTERMEDIO]

No se genera código intermedio por presencia de errores.

Errores léxicos: 0
Errores sintácticos: 4
Errores semánticos: 2

Prueba 3

Interacción con el usuario

```

9 empty ~ __main__ <| |> |:
10 int ~ opcion !
11
12
13 cout <| "Ingrese un numero del 1 al 3:" |> !
14 cin <| opcion |> !
15
16 switch <| opcion |> |:
17     case ~ 1 :
18         cout <| "Usted ingreso la opcion 1" |> !
19         break !
20
21     case ~ 2 :
22         cout <| "Usted ingreso la opcion 2" |> !
23         break !
24
25     case ~ 3 :
26         cout <| "Usted ingreso la opcion 3" |> !
27         break !
28
29     default :
30         cout <| "Opcion no valida" |> !
31
32 :|
33

```

Archivo: Ejemplo.txt

[LÉXICO]

Sin errores léxicos.

[TABLA DE SÍMBOLOS]

[TABLA DE SÍMBOLOS]

Scope: [global]

TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
(sin identificadores)				
Scope: [__main__]				
TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
opcion	VARIABLE	int	25	11
Scope: [switch__main__]				
TOKEN	TIPO	TIPO_DATO	FILA	COLUMNA
(sin identificadores)				

[PARSER]

Sin errores sintácticos.

[ERRORES SEMÁNTICOS]

Sin errores semánticos.

[CÓDIGO INTERMEDIO]

Código guardado en: generated/Codigo3D_Ejemplo.txt

Código MIPS generado en: generated/Ejemplo.asm

El archivo cumple con la gramática y puede ser procesado.

Console

```

1 Ingrese un numero del 1 al 3:
2 1
3 Usted ingreso la opcion 1

```

4. Descripción del Problema

El presente proyecto corresponde a la tercera fase de un compilador para un lenguaje imperativo ligero diseñado para la configuración de chips en sistemas empotrados. En esta fase se aborda la generación de código destino en arquitectura MIPS a partir del código intermedio de tres direcciones producido en el Proyecto II.

El lenguaje fuente es un lenguaje imperativo que soporta tipos de datos primitivos (int, float, bool, char, string), estructuras de control (if/else, while, for, do-while), funciones con parámetros y valor de retorno, arreglos bidimensionales, y operaciones aritméticas, relacionales y lógicas.

Alcance de la Fase

Esta fase comprende las siguientes responsabilidades:

- Recibir como entrada la lista de instrucciones de código intermedio de tres direcciones (3AC) producidas por el Proyecto II.
- Traducir cada instrucción de código intermedio a una o más instrucciones MIPS ensamblador válidas.
- Administrar el frame de activación de cada función: reserva de espacio en el stack, almacenamiento de \$ra y \$fp, y restauración al retornar.
- Gestionar un segmento .data con cadenas literales, arreglos bidimensionales y la constante de salto de línea.
- Generar subrutinas de soporte para operaciones no nativas de MIPS, como la potenciación entera y flotante.
- Producir un archivo .asm ejecutable en QtSpim que sea semánticamente equivalente al código fuente original.

5. Librerías Usadas

El generador de código MIPS está implementado en Java estándar y no requiere dependencias externas más allá del JDK. A continuación se detallan los paquetes y clases utilizadas, junto con su propósito específico dentro del proyecto.

java.io - Entrada/Salida de Archivos

Clase	Uso en el proyecto
-------	--------------------

java.io.FileWriter	Abre el archivo de salida .asm en modo escritura para volcar el código MIPS generado.
java.io.PrintWriter	Escribe línea a línea el contenido de los StringBuilder data y text hacia el archivo .asm de salida.
java.io.IOException	Maneja excepciones en la apertura y escritura del archivo de salida en el método generar().

java.util - Estructuras de Datos

Clase	Uso en el proyecto
java.util.List<String>	Contiene la secuencia ordenada de instrucciones de código intermedio recibidas como entrada.
java.util.ArrayList<String>	Implementación concreta usada en parametrosPendientes para acumular argumentos antes de una llamada a función.
java.util.HashMap<K,V>	Cuatro mapas distintos: offsetVars (offset de cada variable en el stack), tipoVars (tipo de dato), etiquetaArreglo (etiqueta .data de arreglos), tipoArreglos (tipo base de arreglos).

java.util.regex - Análisis de Patrones

Clase	Uso en el proyecto
java.util.regex.Pattern	Usado en Pattern.quote() para escapar operadores como +, *, ^, , &&, >=, etc., al dividir instrucciones con String.split().

StringBuilder Construcción Eficiente del Código

Se emplean dos instancias de `java.lang.StringBuilder` (clase del JDK sin importación explícita) para construir eficientemente el código MIPS antes de escribirlo al disco:

- `data`: acumula la sección `.data` del archivo MIPS (strings, arreglos, `newline`).
- `text`: acumula la sección `.text` con todas las instrucciones ejecutables.

El uso de `StringBuilder` en lugar de concatenación directa de `String` evita la creación de objetos temporales, lo que resulta eficiente cuando se generan miles de líneas de código ensamblador.

Resumen de Dependencias

Paquete	Dependencia externa	Propósito
<code>java.io</code>	Ninguna (JDK)	Escritura del archivo <code>.asm</code>
<code>java.util</code>	Ninguna (JDK)	Estructuras de datos internas
<code>java.util.regex</code>	Ninguna (JDK)	Escape de operadores en <code>split()</code>

6. Análisis de resultados

Área evaluada	Estado	Análisis del resultado
Integración del compilador	Alcanzado	Se logró mantener la base del compilador y utilizarla como punto de partida para la generación de código destino en MIPS.
Generación de código destino	Alcanzado	El sistema genera código MIPS con una estructura básica válida, incluyendo secciones de datos, texto, instrucciones, etiquetas y saltos.
Operaciones básicas	Alcanzado	Traducir asignaciones, operaciones aritméticas, impresiones, lecturas y expresiones simples.
Estructuras de control	Alcanzado	Se logró representar el flujo del programa mediante etiquetas y saltos condicionales o incondicionales en MIPS.
Funciones y retornos	Alcanzado	Se cubrieron casos básicos, pero los escenarios más complejos requieren un

		manejo más completo de pila, registros y retorno de valores.
Manejo de scopes	Alcanzado 80%	Se logro parcialmente ya que en repetición de nombres de variables se sobre escribe su valor. Es un error no visto en la etapa 2 por lo que no se corrigió a falta de tiempo ya que implicaba cambiar el cup, y errores semánticos

7. Detalle de los Algoritmos más Importantes del Proyecto

Algoritmo Principal de Traducción - método traducir()

El corazón del generador es el método traducir(String inst), que implementa una cadena de reconocimiento de patrones sobre cada instrucción de código intermedio. El algoritmo opera en dos pasadas globales:

- Pasada 1 (registrarArregloEnData): recorre todas las instrucciones y emite únicamente las declaraciones var int[]/var float[] en el segmento .data.
- Pasada 2 (traducir): recorre todas las instrucciones en orden y genera el código MIPS correspondiente usando una cascada de condiciones if/else if.

La cascada de reconocimiento sigue esta prioridad:

1. *func <tipo> <nombre>:* → prólogo de función
2. *<etiqueta>:* → etiqueta MIPS (main, main_end, u otras)
3. *param_def <tipo> <nombre>* → almacena parámetro desde registro \$aX/\$fX
4. *print / cout / read / cin* → E/S por syscall
5. *if_true <cond> goto <lbl>* → beq/bne condicional
6. *if_false <cond> goto <lbl>* → beq condicional
7. *goto <lbl>* → salto incondicional j
8. *return <val>* → carga en \$v0/\$f0 + epílogo
9. *param <val>* → acumula en parametrosPendientes
10. *<arr>[i][j] = <val>* → escritura en arreglo 2D
11. *<dest> = <arr>[i][j]* → lectura de arreglo 2D
12. *<dest> = <expr>* → asignación/operación (ver 7.2)

Algoritmo de Traducción de Expresiones

Cuando la instrucción tiene la forma dest = fuente, el método aplica el siguiente algoritmo de clasificación de la expresión del lado derecho:

Si fuente inicia con " → asignación de cadena literal (etiqueta en .data)

Si fuente inicia con "call " → llamada a función (ver 7.4)

Si fuente es variable conocida → copia de registro (\$f0 o \$t0 según tipo)

Si fuente inicia con ! o \$ → operación NOT lógico

Si fuente contiene &&, ||, @, # → operación lógica binaria

Si fuente contiene >=, <=, ==, !=, >, < → comparación relacional

Si fuente contiene +, -, *, /, %, ^ → operación aritmética

Caso base → asignación simple de literal o variable

La detección del tipo float/int se realiza mediante el método esFloat(), que verifica si el operando es un literal con punto decimal, una fracción (/), o una variable registrada con tipo float en tipoVars. Este resultado determina si se usan registros enteros (\$t0–\$t2) o de punto flotante (\$f0–\$f2).

Algoritmo de Gestión del Frame de Activación

Cada función mantiene un frame de activación en el stack de MIPS. El algoritmo de gestión comprende tres fases:

Inicio de función

```
addiu $sp, $sp, -8  # reserva espacio para $ra y $fp
sw $ra, 4($sp)      # salva dirección de retorno
sw $fp, 0($sp)       # salva frame pointer anterior
move $fp, $sp        # establece nuevo frame pointer
```

Cuerpo (variables locales y temporales)

El método obtenerOffset(nombre) mantiene un contador offsetActual que decrementa en 4 por cada nueva variable. Al encontrar una variable no registrada, emite addiu \$sp, \$sp, -4 y registra el offset negativo relativo a \$fp. Las variables se acceden siempre como offset(\$fp), lo que es independiente de cuántas variables se declare después.

Retorno

```
move $sp, $fp        # restaura stack pointer
lw $fp, 0($sp)        # restaura frame pointer anterior
lw $ra, 4($sp)         # restaura dirección de retorno
addiu $sp, $sp, 8      # libera espacio del frame
jr $ra                 # retorna al llamador
```

Al inicio de cada nueva función se invoca reiniciarFrameActual(), que vacía offsetVars y tipoVars para evitar colisiones entre frames de distintas funciones.

Algoritmo de Llamada a Función — Escribir_llamada()

El protocolo de llamada se divide entre dos métodos que trabajan en conjunto. Los parámetros se acumulan en la lista parametrosPendientes mediante instrucciones param <val> procesadas antes de la llamada. Al encontrar dest = call nombreFuncion, el método Escribir_llamada() ejecuta el siguiente algoritmo:

```

contInt = 0, contFloat = 0
Para cada param en parametrosPendientes:
    Si esFloat(param):
        cargarFloat(param, regsFloat[contFloat++]) # $f12, $f14, $f16, $f18
    Si no:
        cargarInt(param, regsInt[contInt++])      # $a0, $a1, $a2, $a3
Limpiar parametrosPendientes
Emitir: jal nombreFuncion
Determinar tipoRetorno desde tipoRetornoFunciones
Si tipoRetorno == float: guardarFloat($f0, dest)
Si no:
    guardarInt($v0, dest)

```

El mapa tipoRetornoFunciones se actualiza tanto al declarar la función (func <tipo> nombre:) como al procesar su instrucción return, lo que permite que las llamadas forward (a funciones aún no declaradas) también resuelvan correctamente el tipo de retorno.

Algoritmo de Acceso a Arreglos Bidimensionales

El acceso a un elemento $arr[i][j]$ de un arreglo con C columnas requiere calcular la dirección lineal. El algoritmo implementado en `calcularDireccionArreglo()` es:

$$dirección = base_arr + (i \times C + j) \times 4$$

Se traduce a MIPS con las siguientes instrucciones:

```

la $t8, arr_nombre      # carga dirección base
# cargar i en $t6, j en $t7
li $t5, C                # número de columnas
mul $t5, $t6, $t5        # i × C
add $t5, $t5, $t7        # i × C + j
sll $t5, $t5, 2          # × 4 (tamaño de word)
add $t8, $t8, $t5        # dirección final del elemento

```

Para escritura se emite `sw/s.s $t0/$f0, 0($t8)`. Para lectura se emite `lw/l.s $t0/$f0, 0($t8)`. La distinción entre arreglos int y float se determina consultando el mapa tipoArreglos.

Algoritmo de Comparación Flotante

Las comparaciones entre valores float no pueden realizarse con las pseudoinstrucciones enteras de MIPS. El método `EscribirComparacionFloat()` implementa el siguiente patrón para cada operador relacional:

```

cargarFloat(izq, $f0)
cargarFloat(der, $f1)
c.lt.s $f0, $f1          # activa flag CC si f0 < f1
bclt cmp_float_true_N    # salta si CC = 1
li $t2, 0                # resultado falso
j cmp_float_fin_N
cmp_float_true_N:

```

```
li $t2, 1          # resultado verdadero
cmp_float_fin_N:
guardarInt($t2, dest)
```

Para los operadores $>$ y $\geq$ se invierten los operandos (\$f1, \$f0) en lugar de \$f0, \$f1, aprovechando que MIPS solo dispone de c.lt.s y c.le.s. El operador $\neq$ usa bc1f (branch if false) en lugar de bc1t. Cada par de etiquetas es único gracias al contador contadorLabelsMips.

Subrutinas de Potenciación

Dado que MIPS no dispone de instrucción nativa de potencia, se generan dos subrutinas de soporte. Ambas implementan multiplicación repetida con un bucle de decremento del exponente.

potencia_int (base entera, exponente entero)

Parámetros: \$a0 = base, \$a1 = exponente. Retorno: \$v0 = $\text{base}^{\text{exponente}}$.

```
li $v0, 1          # acumulador = 1
ble $a1, $zero, fin # si exp <= 0, retornar 1
loop:
    mul $v0, $v0, $a0 # acum = acum * base
    addiu $a1, $a1, -1 # exp--
    bgt $a1, $zero, loop # repetir mientras exp > 0
```

potencia_float (base flotante, exponente flotante)

Parámetros: \$f12 = base, \$f14 = exponente (tratado como entero). Retorno: \$f0 = $\text{base}^{\text{exponente}}$.

El exponente en \$f14 se convierte a entero con cvt.w.s y se mueve a \$t0 con mfc1. El acumulador \$f0 se inicializa en 1.0 cargando el patrón de bits 0x3F800000 (representación IEEE 754 de 1.0). El bucle multiplica \$f0 por \$f12 y decrementa \$t0 hasta cero.

Estas subrutinas solo se emiten cuando el método usaPotencia() detecta la presencia del operador $^$ en alguna instrucción del código intermedio, evitando incluir código muerto en programas que no usan potenciación.